Nullable Types



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



Nullable Type

-What is Nullable Type?

-null coalescing operator (??)

- Benefits of Nullable Type

Nullable Type

What is Nullable Type?

- a value type cannot be assigned a null value. For example, *int i = null* will give you a compile time error.
- recreate it when needed.
- C# 2.0 introduced nullable types that allow you to assign null to value type variables. You can declare nullable types using Nullable<T> where T is a type.

- It provides a way to represent the absence of a value or an undefined state for value types.
- Nullable types are declared by appending a ? symbol to the underlying value type.

```
int? nullableInt = null;
float? nullableFloat = 3.14f;
bool? nullableBool = true;
DateTime? nullableDateTime = DateTime.Now;

Console.WriteLine(nullableInt);          // Output: null
Console.WriteLine(nullableFloat);        // Output: 3.14
Console.WriteLine(nullableBool);         // Output: True
Console.WriteLine(nullableDateTime);     // Output: current date and time
```

null coalescing operator (??)

```
int? i = null;

int j = i ?? 0;

Console.WriteLine(j);
```

Nullable types offer several benefits, including:

- **Nullability for Value Types:** Value types, such as int, float, bool, and DateTime, can be assigned null, indicating the absence of a value or an undefined state.
- **Safety against Null Reference Exceptions:** Nullable types provide safety against null reference exceptions when working with value types. You can perform null checks before accessing the value to prevent potential exceptions.
- **Compatibility with Non-Nullable Types:** Nullable types can be implicitly converted to non-nullable types, and vice versa. This allows you to assign a nullable type to a non-nullable type, given that the nullable type has a value other than null.
- **HasValue and Value Properties:** Nullable types provide the HasValue property, which indicates whether the nullable type has a value or is null. The Value property retrieves the underlying value if HasValue is true.

Here's an example that demonstrates checking for null and accessing the value of a nullable type:

```
int? nullableInt = 42;

if (nullableInt.HasValue)
{
    Console.WriteLine("NullableInt has a value: " + nullableInt.Value);
}
else
{
    Console.WriteLine("NullableInt is null");
}
```

References

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/nullable-value-types

https://www.tutorialsteacher.com/csharp/csharp-nullable-types

https://www.geeksforgeeks.org/c-sharp-nullable-types/

https://www.c-sharpcorner.com/uploadfile/prasoonk/getting-started-with-nullable-types-in-C-Sharp/

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/extension-methods

https://www.javatpoint.com/csharp-nullable